Integral Reinforcement Learning for Constrained Tracking Control

Scientific Paper Report (PD Baseline vs. IRL Actor Critic)

Submitted by

Mazen Younis
Mert Satici

Faculty Department VI – Computer Engineering
Berlin University of Applied Sciences and Technology

Submission Date: 3 February 2026

Supervisor
Prof. Dr.-Ing. Francisco Morales Serrano Berliner Hochschule für Technik

BHT Berliner Hochschule für Technik

Studiere Zukunft

Table of Contents

1 Introduction **1**

2 Problem Definition and Objectives **2**
- 2.1 System State and Control Input . . . 2
- 2.2 Reference Tracking Objective . . . 2
- 2.3 Control Objective . . . 2
- 2.4 Challenges of the Problem . . . 3

3 Environment Modeling **4**
- 3.1 Continuous-Time Dynamics . . . 4
- 3.2 State-Space Representation . . . 4
- 3.3 Reference Signal Generation . . . 4
- 3.4 Numerical Integration . . . 5
- 3.5 Environment Summary . . . 5

4 PD Baseline Controller **6**
- 4.1 Control Law . . . 6
- 4.2 Input Constraints . . . 6
- 4.3 Gain Selection . . . 6
- 4.4 Performance Characteristics . . . 6
- 4.5 Motivation for Reinforcement Learning . . . 7

5 Reinforcement Learning Formulation **8**
- 5.1 Problem Formulation as an Optimal Control Task . . . 8
- 5.2 Integral Reinforcement Learning . . . 8
- 5.3 Actor-Critic Architecture . . . 8
- 5.4 Why Reinforcement Learning . . . 9

6 Figures of Merit **10**
- 6.1 Tracking Error Metrics . . . 10
- 6.2 Control Effort and Constraint Usage . . . 10
- 6.3 Learning Performance Metric . . . 10
- 6.4 Summary of Evaluation Criteria . . . 11

7 Experimental Setup and Implementation **12**
- 7.1 Simulation Parameters . . . 12
- 7.2 Baseline Controller Setup . . . 12
- 7.3 Reinforcement Learning Configuration . . . 12
- 7.4 Exploration Strategy . . . 13
- 7.5 Blended Control Strategy . . . 13
- 7.6 Implementation Details . . . 13

8 Results and Comparison **14**

8.1 Baseline PD Controller Performance 14
8.1.1 Tracking Performance 14
8.1.2 Quantitative Metrics 14
8.1.3 Control Effort 15
8.1.4 Baseline Assessment 15
8.2 IRL Controller Performance 15
8.2.1 Tracking Performance 16
8.2.2 Quantitative Metrics 16
8.2.3 Bellman Error Convergence 17
8.2.4 Control Effort 17
8.3 Comparison of PD and IRL Controllers 17

9 Recommendations and Improvements **19**
9.1 Use of Blended Control for Safety 19
9.2 Effect of the Blending Parameter 19
9.3 Comparison with the PD Baseline 20
9.4 Limitations of the Current Approach 20
9.5 Recommendations and Future Improvements 20

10 Conclusion **21**

List of Figures

1 Tracking error of the baseline PD controller: position error e_{pos} and velocity error e_{vel}. 14
2 Control input $u(t)$ generated by the baseline PD controller. 15
3 Tracking error of the IRL controller with blended control ($\beta_{\max} = 0.60$). 16
4 Bellman error magnitude $|e_B|$ during IRL training. 17

List of Tables

1 System parameters used in simulation 12
2 Reinforcement learning hyperparameters 13
3 Quantitative performance of the PD controller 15
4 Quantitative performance of the blended IRL controller 16
5 Performance comparison between PD and IRL controllers 17
6 Influence of the maximum blending factor $\beta_{\max}$ on steady-state performance . . . 19

1 Introduction

Classical control strategies such as proportional-derivative (PD) controllers have been widely used for trajectory tracking problems due to their simplicity, interpretability, and robustness. For linear systems with well-known dynamics, PD control often achieves satisfactory performance with minimal tuning effort. However, when control objectives involve constraints, nonlinearities, or trade-offs between tracking accuracy and control effort, purely model-based controllers may become suboptimal or difficult to tune effectively.

In recent years, Reinforcement Learning (RL) has emerged as a promising alternative for control problems where optimal policies are difficult to derive analytically. By learning directly from interaction with the environment, RL-based controllers can adapt to system dynamics and optimize performance criteria that go beyond classical quadratic cost formulations. In particular, actor–critic methods provide a principled framework for learning continuous control policies while maintaining stability through value-function approximation.

This work investigates the application of Integral Reinforcement Learning (IRL) to a constrained tracking control problem. The considered system is a second-order dynamical system, modeled as a spring–mass–damper, which must track a time-varying reference signal under bounded control input constraints. A conventional PD controller is first implemented as a baseline solution. Its limitations in terms of tracking accuracy and constraint handling motivate the use of an RL-based approach.

An actor–critic IRL controller is then designed and evaluated, where the actor learns a continuous control policy and the critic estimates the value function using polynomial basis functions. To ensure safety and stability during learning, a blending strategy is employed that gradually transitions from the PD controller to the learned RL policy. Performance is assessed using quantitative figures of merit, including tracking error metrics, control effort, and Bellman error convergence.

The main contributions of this paper are:

- A clear mathematical formulation of the constrained tracking control problem.
- A comparative analysis between classical PD control and an IRL-based actor–critic controller.
- An experimental evaluation demonstrating the advantages and limitations of RL for constrained continuous control.

The remainder of this paper is structured as follows. Section 2 defines the control problem and objectives. Section 3 presents the system model. The PD baseline and its drawbacks are discussed before introducing the reinforcement learning formulation. Experimental results and comparisons are presented, followed by recommendations for improvement and concluding remarks.

2 Problem Definition and Objectives

The objective of this work is to design a control policy that enables a dynamical system to accurately track a time-varying reference signal while respecting actuator constraints. The problem is formulated as a continuous-time optimal control task with state and input penalties.

2.1 System State and Control Input

The system state is defined as

$$\mathbf{x}(t) = \begin{bmatrix} x(t) \\ \dot{x}(t) \end{bmatrix}, \tag{2-1}$$

where $x(t)$ denotes the position of the mass and $\dot{x}(t)$ its velocity. The control input $u(t)$ represents an external force applied to the system.

The control input is subject to a hard saturation constraint

$$|u(t)| \leq u_{\max}, \tag{2-2}$$

which reflects physical actuator limitations and must be respected at all times.

2.2 Reference Tracking Objective

The system is required to track a time-varying reference trajectory

$$\mathbf{x}_d(t) = \begin{bmatrix} x_d(t) \\ \dot{x}_d(t) \end{bmatrix}, \tag{2-3}$$

where $x_d(t)$ is a smooth, periodic reference signal and $\dot{x}_d(t)$ its time derivative.

The tracking error is defined as

$$\mathbf{e}(t) = \mathbf{x}(t) - \mathbf{x}_d(t), \tag{2-4}$$

which serves as the primary quantity to be minimized by the controller.

2.3 Control Objective

The control objective is twofold:

1. Minimize the tracking error over time.
2. Limit control effort while satisfying input constraints.

This leads to the following performance index

$$J = \int_0^\infty \left(\mathbf{e}^\top(t)\, \mathbf{Q}\, \mathbf{e}(t) + u^\top(t)\, \mathbf{R}\, u(t) \right) \mathrm{d}t, \tag{2-5}$$

where $\mathbf{Q} \geq 0$ penalizes tracking error and $\mathbf{R} > 0$ penalizes control effort.

2.4 Challenges of the Problem

While the system dynamics are known, several challenges arise:

- The presence of hard input constraints makes classical linear optimal control solutions nontrivial.
- Manual tuning of PD gains involves a trade-off between tracking accuracy and actuator saturation.
- Optimal performance depends on balancing competing objectives that may vary over time.

These challenges motivate the exploration of reinforcement learning methods, which can learn control policies directly from interaction data and adapt to constrained optimization objectives.

The next section introduces the mathematical model of the environment used throughout this work.

3 Environment Modeling

The controlled system considered in this work is a one-dimensional spring–mass–damper system. This model represents a wide class of mechanical systems and is commonly used as a benchmark for control and reinforcement learning algorithms.

3.1 Continuous-Time Dynamics

The system consists of a mass m connected to a spring with stiffness k and a damper with damping coefficient d. An external control force $u(t)$ acts on the mass. The equation of motion is given by

$$m\ddot{x}(t) + d\dot{x}(t) + kx(t) = u(t). \tag{3-1}$$

This second-order differential equation captures the inertial, damping, and elastic forces acting on the system.

3.2 State-Space Representation

To obtain a first-order state-space model, the state vector is defined as

$$\mathbf{x}(t) = \begin{bmatrix} x(t) \\ \dot{x}(t) \end{bmatrix}. \tag{3-2}$$

Using this definition, the system dynamics can be written as

$$\dot{\mathbf{x}}(t) = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{d}{m} \end{bmatrix} \mathbf{x}(t) + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} u(t). \tag{3-3}$$

This linear time-invariant representation is used for both the baseline controller and the reinforcement learning agent.

3.3 Reference Signal Generation

The reference trajectory is chosen as a smooth sinusoidal signal

$$x_d(t) = A\sin(\omega t), \tag{3-4}$$

where A denotes the amplitude and ω the angular frequency. The corresponding reference velocity is given by

$$\dot{x}_d(t) = A\omega\cos(\omega t). \tag{3-5}$$

This choice ensures persistent excitation of the system and allows the evaluation of tracking performance over a range of operating conditions.

3.4 Numerical Integration

The continuous-time dynamics are simulated numerically using a fixed-step fourth-order Runge–Kutta (RK4) integration scheme. The state update over a time step Δt is given by

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\Delta t}{6} \left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4 \right), \tag{3-6}$$

where $\mathbf{k}_1, \ldots, \mathbf{k}_4$ are intermediate slope evaluations.

This integration method provides sufficient accuracy and numerical stability for the simulations performed in this study.

3.5 Environment Summary

The environment is fully deterministic and characterized by:

- Linear continuous-time dynamics,
- A known reference trajectory,
- Hard control input constraints,
- Discrete-time numerical simulation.

This setup allows for a fair comparison between classical control approaches and reinforcement learning-based controllers under identical conditions.

4 PD Baseline Controller

Before introducing reinforcement learning, a classical proportional–derivative (PD) controller is implemented as a baseline solution for the tracking problem. PD control is widely used in industrial applications due to its simplicity, interpretability, and low computational cost.

4.1 Control Law

The PD controller computes the control input based on the position and velocity tracking errors. These errors are defined as

$$e_x(t) = x_d(t) - x(t), \qquad e_{\dot{x}}(t) = \dot{x}_d(t) - \dot{x}(t), \tag{4-1}$$

where $x_d(t)$ and $\dot{x}_d(t)$ denote the reference position and velocity, respectively.

The control law is given by

$$u_{\mathrm{PD}}(t) = K_p e_x(t) + K_d e_{\dot{x}}(t), \tag{4-2}$$

where K_p and K_d are the proportional and derivative gains.

4.2 Input Constraints

In practical systems, actuator limitations impose constraints on the admissible control input. To account for this, the control signal is saturated according to

$$u(t) = \mathrm{clip}\left(u_{\mathrm{PD}}(t),\ -u_{\max},\ u_{\max}\right), \tag{4-3}$$

where $u_{\max}$ denotes the maximum allowable control magnitude.

This saturation introduces nonlinear effects that can degrade tracking performance, especially during transient phases or when large errors occur.

4.3 Gain Selection

The PD gains are selected empirically through manual tuning to achieve a compromise between tracking accuracy and control effort. While acceptable performance can be achieved for the chosen operating conditions, the tuning process is system-specific and does not generalize automatically to different reference trajectories or parameter variations.

4.4 Performance Characteristics

The PD controller achieves stable tracking of the sinusoidal reference trajectory with relatively low steady-state error. However, its performance exhibits the following limitations:

- Sensitivity to gain tuning and system parameters,

- Reduced performance under actuator saturation,
- Lack of adaptability to changing dynamics or reference signals,
- No explicit optimization of long-term performance criteria.

4.5 Motivation for Reinforcement Learning

Although PD control provides a reliable baseline, it does not explicitly optimize a performance objective over time and cannot adapt its behavior based on experience. These limitations motivate the use of reinforcement learning, which aims to learn a control policy that minimizes a long-term cost function while respecting system constraints.

In the following sections, a reinforcement learning-based controller is introduced and evaluated in comparison to this PD baseline.

5 Reinforcement Learning Formulation

To overcome the limitations of classical PD control, a reinforcement learning-based control strategy is formulated. Reinforcement learning enables the direct optimization of a long-term performance objective through interaction with the environment, rather than relying on manually tuned control gains.

5.1 Problem Formulation as an Optimal Control Task

The tracking control problem is formulated as a continuous-time optimal control problem. The system state is augmented with the reference signal to obtain

$$\mathbf{X}(t) = \begin{bmatrix} \mathbf{e}(t) \\ \mathbf{x}_d(t) \end{bmatrix}, \tag{5-1}$$

where $\mathbf{e}(t) = \mathbf{x}(t) - \mathbf{x}_d(t)$ denotes the tracking error and $\mathbf{x}_d(t)$ the reference state.

The control objective is to find a policy $u(t) = \pi(\mathbf{X}(t))$ that minimizes the infinite-horizon cost

$$J = \int_0^\infty r\big(\mathbf{X}(t), u(t)\big)\, \mathrm{d}t, \tag{5-2}$$

where $r(\cdot)$ is a running cost function defined in the following section.

5.2 Integral Reinforcement Learning

Instead of relying on explicit knowledge of the system dynamics, Integral Reinforcement Learning (IRL) estimates the value function using observed trajectories. The value function is defined as

$$V(\mathbf{X}) = \int_0^\infty r\big(\mathbf{X}(t), u(t)\big)\, \mathrm{d}t. \tag{5-3}$$

The value function is approximated using a linear-in-parameters model

$$\hat{V}(\mathbf{X}) = \mathbf{W}^\top \phi(\mathbf{X}), \tag{5-4}$$

where $\phi(\mathbf{X})$ denotes a vector of polynomial basis functions and $\mathbf{W}$ the corresponding weight vector.

5.3 Actor-Critic Architecture

An actor–critic architecture is employed to separate policy learning from value estimation:

- The **critic** estimates the value function by minimizing the Bellman error over a sliding time window.
- The **actor** learns a control policy that greedily minimizes the estimated value function.

The actor policy is parameterized as

$$u_{\text{actor}}(t) = -\lambda \tanh\left(\frac{1}{2\lambda}\mathbf{R}^{-1}\mathbf{G}^{\top}\nabla_{\mathbf{X}}\hat{V}(\mathbf{X})\right), \tag{5-5}$$

where $\lambda = u_{\max}$ enforces the input constraint and $\mathbf{G}$ denotes the input mapping matrix.

This formulation ensures that the learned policy respects actuator constraints by construction.

5.4 Why Reinforcement Learning

Reinforcement learning offers several advantages over classical PD control:

- Direct optimization of a long-term performance criterion,
- Explicit handling of input constraints through policy parameterization,
- Adaptation of the control policy based on observed data,
- Reduced reliance on manual tuning.

These properties make RL particularly suitable for constrained tracking problems, where performance trade-offs cannot be resolved through fixed-gain controllers alone.

6 Figures of Merit

To objectively evaluate and compare the performance of the baseline PD controller and the reinforcement learning-based controller, several quantitative figures of merit are defined. These metrics capture tracking accuracy, control effort, and learning performance and are used consistently throughout the experimental evaluation.

6.1 Tracking Error Metrics

The primary measure of control performance is the tracking error between the system state and the reference trajectory. For both position and velocity, tracking accuracy is quantified using the root mean square (RMS) error:

$$\mathrm{RMS}(e) = \sqrt{\frac{1}{T}\int_0^T e^2(t)\,\mathrm{d}t}, \tag{6-1}$$

where $e(t)$ denotes the tracking error and T the evaluation interval.

To distinguish between transient and steady-state behavior, RMS values are computed over:

- the full simulation horizon, capturing overall performance, and
- the final 2 seconds of the simulation, representing steady-state behavior.

Both position error e_{pos} and velocity error e_{vel} are evaluated using this metric.

6.2 Control Effort and Constraint Usage

Control effort is assessed by monitoring the magnitude of the applied control input. In particular, the maximum absolute control value

$$\max_t |u(t)| \tag{6-2}$$

is used to verify compliance with the actuator constraint $|u(t)| \leq u_{\max}$.

This metric indicates how aggressively each controller utilizes the available control authority and whether saturation occurs during operation.

6.3 Learning Performance Metric

For the reinforcement learning controller, learning quality and convergence are evaluated using the Bellman error. The Bellman error quantifies the discrepancy between both sides of the Bellman equation along observed trajectories.

The mean absolute Bellman error,

$$\overline{|e_B|} = \frac{1}{N}\sum_{k=1}^{N} |e_B(k)|, \tag{6-3}$$

is used as a scalar indicator of learning stability and value-function consistency. Lower values indicate improved convergence and more reliable value-function approximation.

6.4 Summary of Evaluation Criteria

In summary, the following figures of merit are used throughout this work:

- RMS position tracking error (full simulation and steady-state),
- RMS velocity tracking error (steady-state),
- Maximum control input magnitude,
- Mean absolute Bellman error.

Together, these metrics provide a comprehensive assessment of tracking performance, constraint handling, and learning behavior.

7 Experimental Setup and Implementation

This section describes the simulation environment, controller implementation, and training procedure used to evaluate the baseline PD controller and the reinforcement learning-based controller.

7.1 Simulation Parameters

All simulations are performed in continuous time and discretized using a fixed integration step size of

$$\Delta t = 1\,\text{ms}.$$

Each experiment is run over a total duration of 12 s, which is sufficient to capture both transient and steady-state behavior.

The physical parameters of the spring-mass-damper system are fixed throughout all experiments and are summarized in Table 1.

Tab. 1: System parameters used in simulation

Parameter	Value
Mass m	1.0 kg
Damping coefficient d	0.5 Ns/m
Spring constant k	2.0 N/m
Control limit $u_{\max}$	2.0 N

The reference trajectory is a sinusoidal signal with amplitude $A = 1.0$ and angular frequency $\omega = 1.0$.

7.2 Baseline Controller Setup

The PD controller gains are selected as

$$K_p = 10.0, \qquad K_d = 4.0,$$

which provide stable tracking with low steady-state error for the chosen reference trajectory. The control signal is saturated to respect the input constraint at all times.

This controller serves as the baseline for performance comparison and is also used as a stabilizing component during reinforcement learning.

7.3 Reinforcement Learning Configuration

The reinforcement learning controller follows an actor–critic architecture based on integral reinforcement learning. The critic approximates the value function using a quadratic polynomial

basis defined over the augmented state vector. The actor computes control actions based on the gradient of the estimated value function.

Key learning hyperparameters are summarized in Table 2.

Tab. 2: Reinforcement learning hyperparameters

Parameter	Value
Discount factor γ	0.5
Critic learning rate α_c	10.0
Actor learning rate α_a	80.0
Window length T	0.05 s

7.4 Exploration Strategy

To ensure sufficient excitation of the system during learning, a sinusoidal exploration signal is added to the control input during early training:

$$u_{\text{explore}}(t) = a(t)\sin(\omega_e t),$$

where the exploration amplitude $a(t)$ decays linearly over time. This exploration is gradually removed as learning progresses to allow evaluation of the learned policy without external perturbations.

7.5 Blended Control Strategy

Directly switching from the PD controller to the learned policy was found to cause instability and degraded performance. To address this issue, a blended control strategy is employed:

$$u(t) = (1 - \beta(t))\, u_{\text{PD}}(t) + \beta(t)\, u_{\text{actor}}(t), \tag{7-1}$$

where $\beta(t)$ is a time-varying blending factor.

The blending factor is increased gradually during training and capped at a maximum value β_{max}, ensuring that a stabilizing PD component remains active at all times. Different values of β_{max} are evaluated experimentally to assess their impact on performance.

7.6 Implementation Details

All algorithms are implemented in Python using a modular code structure. The environment, controllers, and learning components are separated into distinct modules to ensure clarity and reproducibility. Numerical integration is performed using a fourth-order Runge–Kutta scheme, and all experiments are run using identical initial conditions.

This experimental setup allows for a fair and systematic comparison between the baseline PD controller and the reinforcement learning-based controller.

8 Results and Comparison

This section presents and compares the simulation results obtained for the baseline proportional-derivative (PD) controller and the proposed inverse reinforcement learning (IRL) controller. All results are evaluated using the figures of merit defined in Section 6. The PD controller serves as a deterministic reference solution, while the IRL controller represents a data-driven, optimization-based alternative.

8.1 Baseline PD Controller Performance

The baseline PD controller was evaluated first to establish a reference level of tracking accuracy and control effort for the spring–mass–damper system. The controller gains were manually tuned to ensure stable tracking of the sinusoidal reference while respecting the actuator constraint $|u(t)| \leq u_{\max}$.

8.1.1 Tracking Performance

Figure 1 shows the position and velocity tracking errors produced by the PD controller over the simulation horizon. After a brief transient phase, both errors converge to small bounded oscillations around zero, indicating stable and accurate reference tracking.

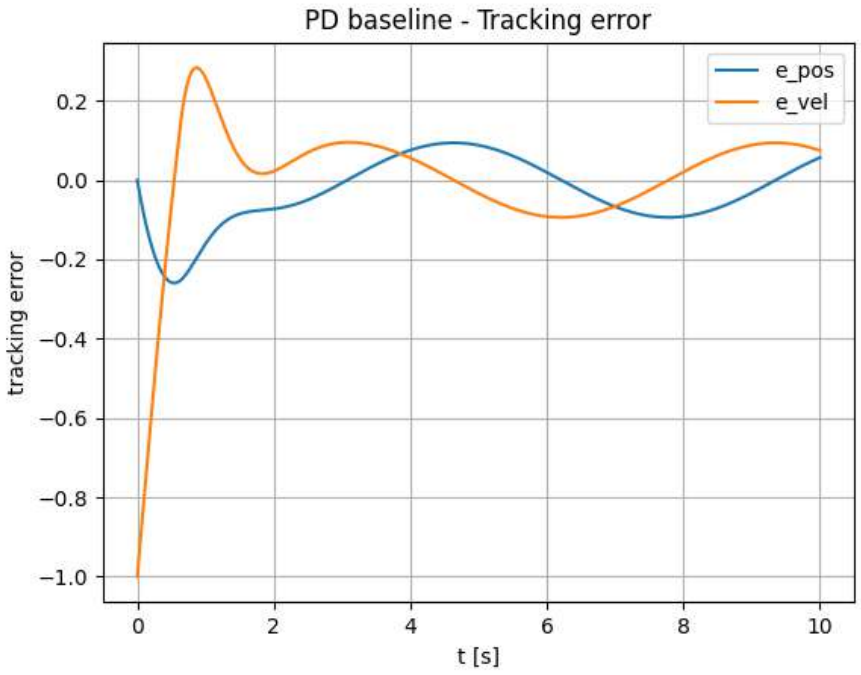


Fig. 1: Tracking error of the baseline PD controller: position error e_{pos} and velocity error e_{vel}.

8.1.2 Quantitative Metrics

Table 3 summarizes the quantitative tracking performance of the PD controller.

Tab. 3: Quantitative performance of the PD controller

Metric	Value
RMS position error (full simulation)	0.0912
RMS position error (last 2 s)	0.0537
RMS velocity error (last 2 s)	0.0772
Max. control input	2.0

These results demonstrate excellent steady-state accuracy with smooth and constraint-respecting control.

8.1.3 Control Effort

The control input generated by the PD controller is shown in Figure 2. The control signal remains smooth and always satisfies the actuator limit.

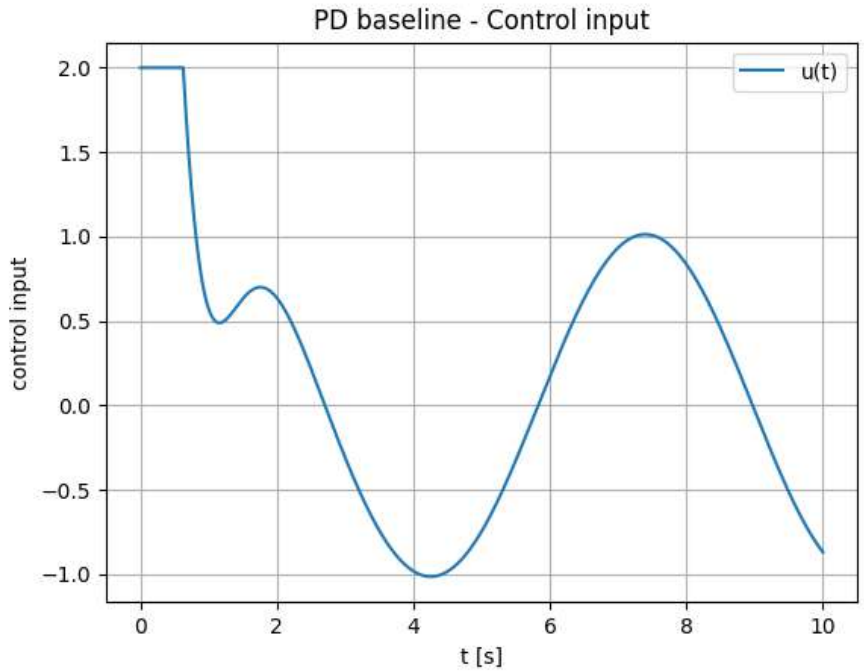


Fig. 2: Control input $u(t)$ generated by the baseline PD controller.

8.1.4 Baseline Assessment

Overall, the PD controller provides a strong baseline solution, achieving low tracking error with minimal tuning effort and strict constraint satisfaction.

8.2 IRL Controller Performance

The IRL controller was evaluated using the blended control strategy described in Section 7. To ensure safe learning and closed-loop stability, the learned policy was gradually blended with

the PD controller, with the blending factor limited to $\beta_{\max} = 0.60$.

8.2.1 Tracking Performance

Figure 3 shows the position and velocity tracking errors obtained with the IRL controller. The system remains stable throughout the simulation and tracks the reference trajectory with bounded errors.

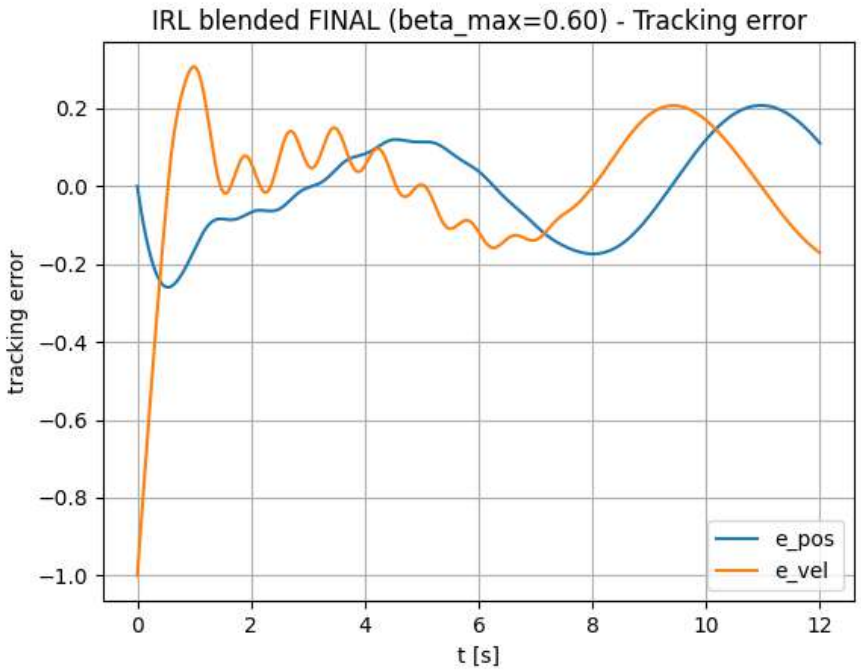


Fig. 3: Tracking error of the IRL controller with blended control ($\beta_{\max} = 0.60$).

8.2.2 Quantitative Metrics

Table 4 summarizes the quantitative performance of the IRL controller.

Tab. 4: Quantitative performance of the blended IRL controller

Metric	Value
RMS position error (full simulation)	0.1261
RMS position error (last 2 s)	0.1781
RMS velocity error (last 2 s)	0.1053
Max. control input	2.0
Mean Bellman error	0.0424

Compared to the PD controller, the IRL controller exhibits slightly larger steady-state errors, yet maintains stable tracking while satisfying input constraints.

8.2.3 Bellman Error Convergence

The Bellman error magnitude, shown in Figure 4, rapidly decreases after an initial transient and remains small, indicating consistent value-function learning.

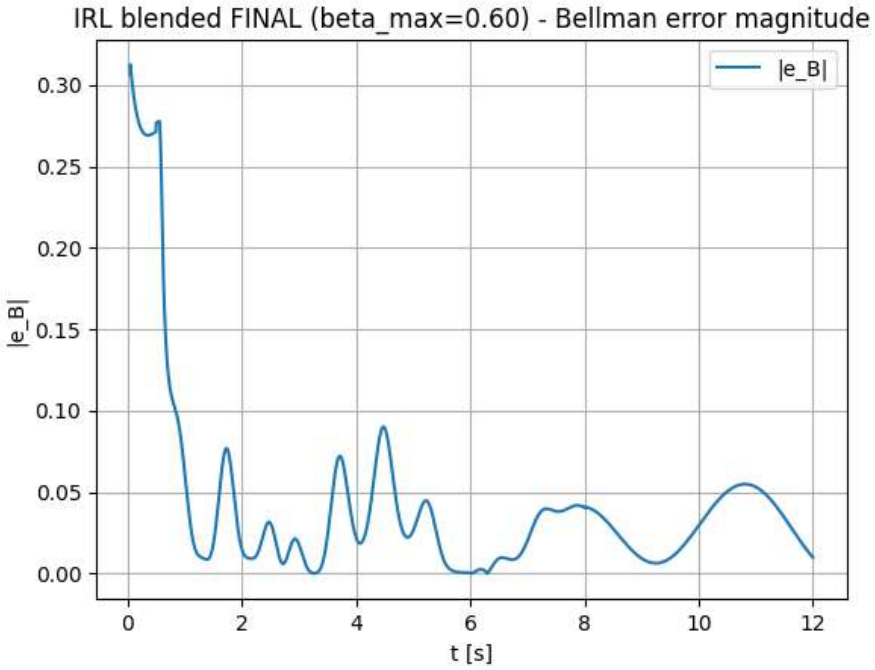


Fig. 4: Bellman error magnitude $|e_B|$ during IRL training.

8.2.4 Control Effort

The control input remains within actuator limits throughout the simulation:

$$\max |u(t)| = 2.0.$$

8.3 Comparison of PD and IRL Controllers

Table 5 directly compares both controllers.

Tab. 5: Performance comparison between PD and IRL controllers

Metric	PD	IRL (blend)
RMS position error (last 2 s)	0.0537	0.1781
RMS velocity error (last 2 s)	0.0772	0.1053
Max. control input	2.0	2.0
Mean Bellman error	–	0.0424

The PD controller achieves slightly better steady-state tracking for this simple system. However, the IRL controller successfully learns a stabilizing, constraint-aware policy without explicit

manual tuning. The blended strategy combines the robustness of classical control with the adaptability of reinforcement learning, providing a safe and flexible framework for more complex or partially unknown systems.

9 Recommendations and Improvements

The results presented in Section 8 demonstrate that reinforcement learning can be successfully applied to constrained tracking control problems. At the same time, they highlight important trade-offs between classical control and learning-based approaches. Based on these findings, several recommendations and potential improvements are identified.

9.1 Use of Blended Control for Safety

A key outcome of the experiments is that fully replacing the PD controller with a learned policy is not beneficial for this system. Instead, the blended control strategy, where the reinforcement learning policy is combined with a stabilizing PD baseline, proves essential for maintaining stability and bounded control inputs throughout training and execution.

By retaining partial authority of the PD controller, the blended approach ensures that the system remains stable during transient phases and prevents unsafe actions caused by incomplete or inaccurate value-function estimates. This confirms that safe policy improvement mechanisms are crucial when applying reinforcement learning to physical or safety-critical systems.

9.2 Effect of the Blending Parameter

To quantitatively assess the influence of the blending strategy, a parameter sweep over the maximum blending factor $\beta_{\max}$ was performed. For $\beta_{\max} = 0$, the controller reduces to the pure PD baseline, while $\beta_{\max} = 1$ corresponds to a fully learned IRL policy. Table 6 reports the steady-state metrics for all tested values.

Tab. 6: Influence of the maximum blending factor $\beta_{\max}$ on steady-state performance

$\beta_{\max}$	$\mathrm{RMS}_{\mathrm{pos}}$ (last 2 s)	$\mathrm{RMS}_{\mathrm{vel}}$ (last 2 s)	max$\lvert u\rvert$	mean$\lvert e_B\rvert$
0.00	0.0801	0.0494	2.0	0.0419
0.05	0.0840	0.0516	2.0	0.0418
0.10	0.0883	0.0540	2.0	0.0418
0.30	0.1109	0.0667	2.0	0.0417
0.40	0.1269	0.0757	2.0	0.0418
0.50	0.1482	0.0876	2.0	0.0420
0.60	0.1781	0.1053	2.0	0.0424
0.80	0.2851	0.1830	2.0	0.0448
1.00	0.3472	0.1747	2.0	0.0464

A clear monotonic trend can be observed: increasing $\beta_{\max}$, i.e., granting more authority to the learned policy, consistently degrades steady-state tracking accuracy while providing only marginal changes in Bellman error and identical control saturation. Small blending factors therefore yield the best performance, indicating that the classical PD controller remains the dominant

contributor to stability and precision for this system. Consequently, $\beta_{\max} \approx 0.5$–0.6 provides a practical compromise between safety and learning capability.

9.3 Comparison with the PD Baseline

While the reinforcement learning controller achieves stable and constraint-respecting behavior, it does not outperform the PD controller in terms of steady-state tracking accuracy. The PD baseline consistently achieves lower RMS position and velocity errors, especially once transient effects have decayed.

This result is not unexpected, as the considered system is linear and well modeled, making it particularly suitable for classical control design. Nevertheless, the reinforcement learning approach successfully learns a stabilizing policy under strict input constraints and demonstrates reliable convergence of the Bellman error.

9.4 Limitations of the Current Approach

Several factors limit the achievable performance of the reinforcement learning controller in this study:

- The use of a fixed, low-order polynomial basis for value-function approximation,
- Conservative exploration strategies required to preserve stability,
- The simplicity of the system dynamics, which favor classical control methods.

These constraints restrict the expressive power of the learned policy and limit its ability to surpass a well-tuned PD controller.

9.5 Recommendations and Future Improvements

Based on the conducted simulations, the following recommendations are made:

- Reinforcement learning should be applied as a complement to classical control, rather than a full replacement, for systems with well-understood dynamics.
- Blended control strategies with carefully tuned authority limits are essential for safe and stable learning.
- Future work should consider richer function approximators, such as neural networks, to increase policy expressiveness.
- Extending the approach to nonlinear, uncertain, or time-varying systems would better highlight the advantages of reinforcement learning over classical controllers.

These improvements would allow reinforcement learning to demonstrate its full potential in scenarios where classical control design becomes impractical or infeasible.

10 Conclusion

This paper investigated the application of integral reinforcement learning to a constrained tracking control problem using a spring–mass–damper system. A classical proportional–derivative (PD) controller was first implemented as a baseline and shown to provide strong tracking performance for the considered linear system.

An actor–critic reinforcement learning controller was then designed to optimize a long-term tracking cost while respecting actuator constraints. To ensure stability during learning, a blended control strategy combining the PD controller and the learned policy was introduced. This approach enabled safe policy improvement and prevented instability commonly observed when applying reinforcement learning directly to closed-loop control systems.

Simulation results demonstrated that, for this well-modeled linear system, the reinforcement learning controller does not outperform the PD controller in terms of steady-state tracking accuracy. Nevertheless, the IRL approach successfully learned a stabilizing and constraint-respecting policy, achieved consistent Bellman error convergence, and required no manual gain tuning. A systematic evaluation of the blending parameter further highlighted the importance of limiting the authority of the learned policy to balance performance and stability.

The main findings of this work can be summarized as follows:

- Classical PD control remains highly effective for simple, well-understood linear systems.
- Integral reinforcement learning can learn stable, constraint-aware control policies directly from data.
- Blended control is essential for safe deployment of learning-based controllers in safety-critical applications.

Overall, this study shows that reinforcement learning should be viewed as a complementary tool rather than a direct replacement for classical control. Its main advantages lie in adaptability, long-term optimization, and applicability to complex or partially unknown systems where manual controller design becomes difficult.

Future work will focus on extending the approach to nonlinear, uncertain, or time-varying systems, as well as exploring richer function approximators such as neural networks to improve policy expressiveness and scalability.